McStas

Simulating Polarized Neutron Scattering Experiments and Equipment with McStas

Peter Willendrup, DTU Physics & ESS DMSC

Presentation content

- Polarisation in McStas - how is it done
  - Polarised monitors
  - Precession algorithm
- Polarising by spin up / down reflectivities
- An overview of polarisation comps
- An overview of polarisation instrument examples
- A quick look at 3 of these
- Words of warning

McStas "particle" model

Neutron ray/package:

Weight: (p) # neutrons left in the package
Position: (x, y, z)

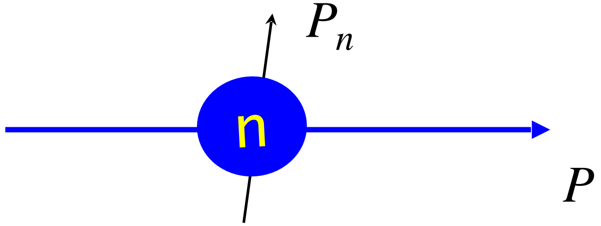
Velocity: (v_x, v_y, v_z)

Polarization: (s_x, s_y, s_z)

Time: (t)

"Within one ray"

$$P_n = \frac{1}{p_n}\sum_i^p P_{i,n};\ n = raynumber$$



"Over the beam"

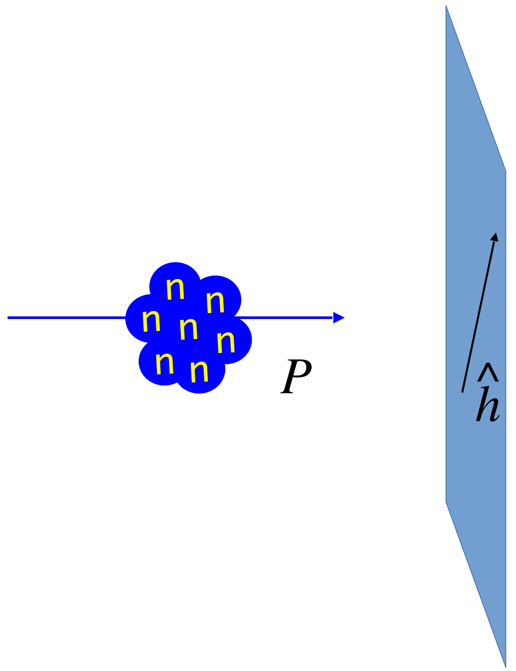
$$P = \frac{1}{N}\sum_{n=0}^{N} P_n$$

$$P_{i,n} = 2\left(\left\langle \hat{s}_{x,i}\right\rangle \hat{i}_{x,i} + \left\langle \hat{s}_{y,i}\right\rangle \hat{i}_{y,i} + \left\langle \hat{s}_{z,i}\right\rangle \hat{i}_{z,i}\right)$$

From G. Williams: *"Polarized neutrons", Oxford Science Publ.*, 1988

McStas detectors/monitors

Monitoring: How and What do we monitor?

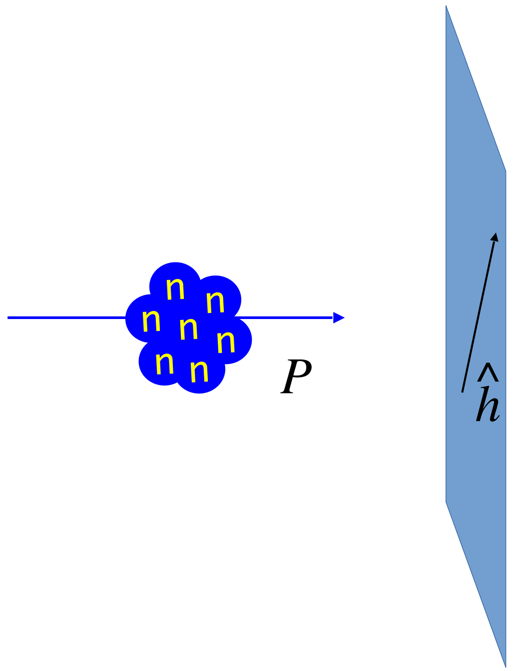


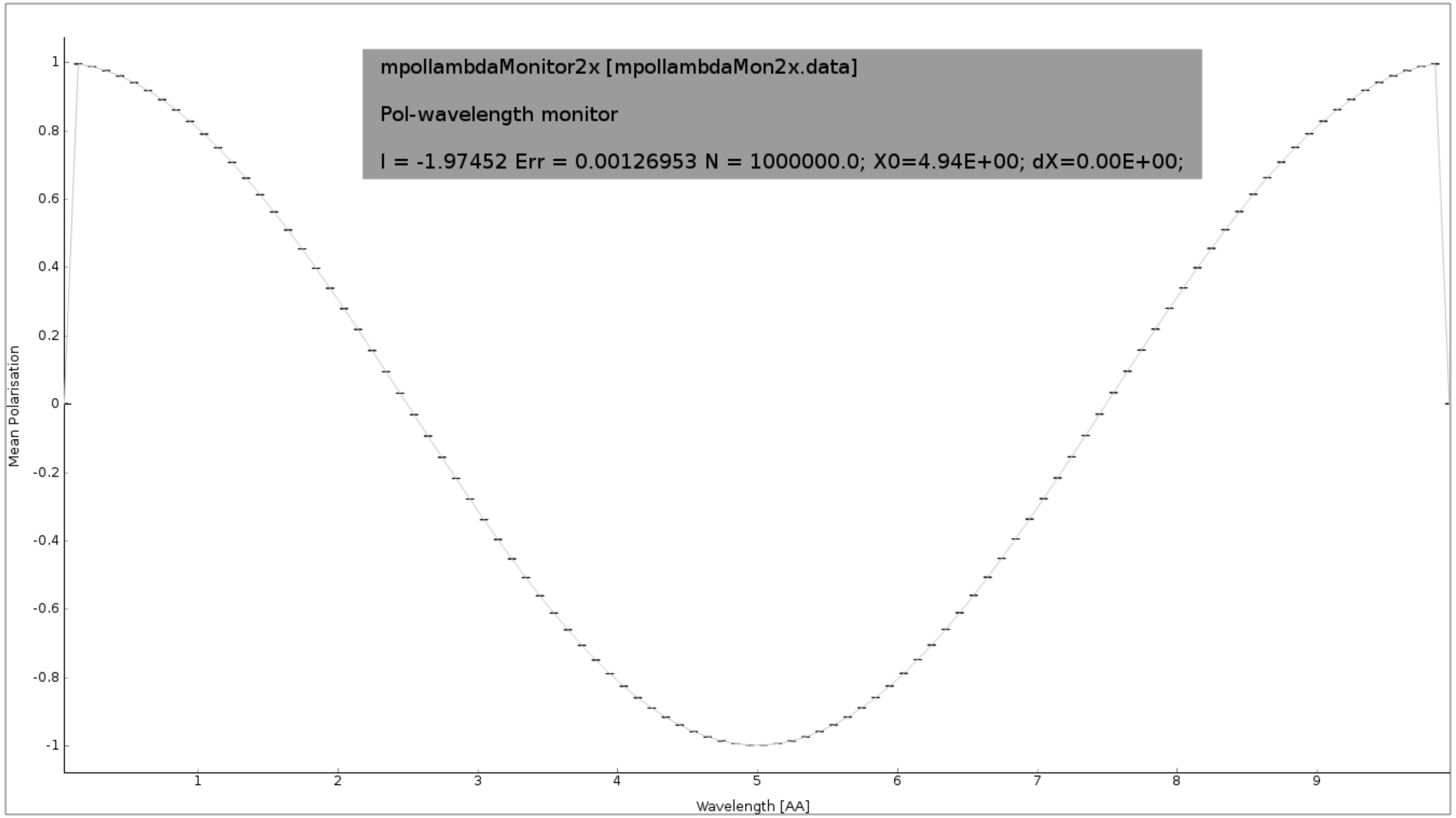
$$P_{\hat{h}} = \frac{\sum_{n=0}^{N} p_n P_n \cdot \hat{h}}{\sum_{n}^{N} p_n}$$

(Statistic-) weighted polarisation along chosen monitoring direction

McStas detectors/monitors

Monitoring: How and What do we monitor?

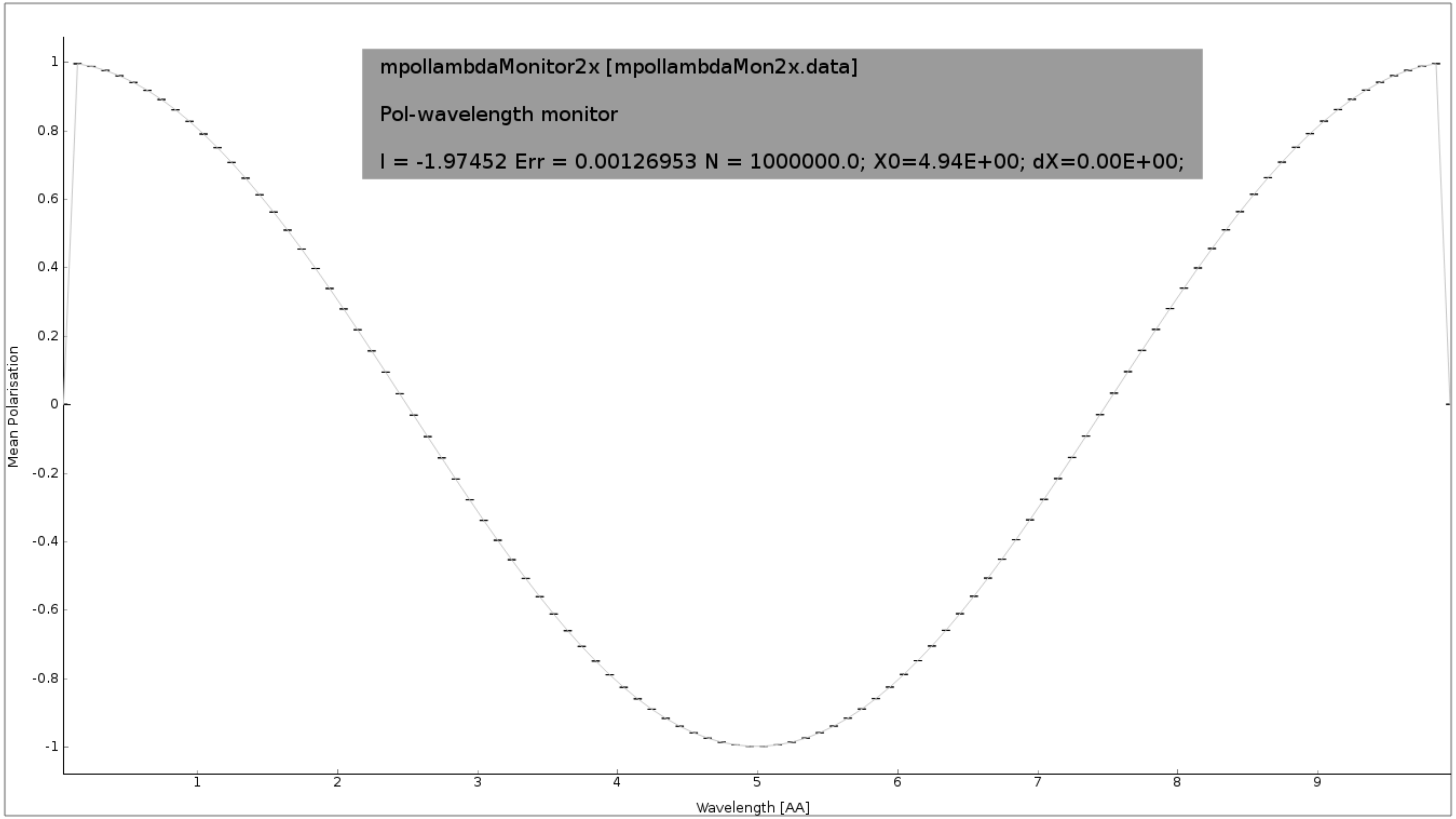




E.g. polarisation along $\hat{h}$ as fct. of wavelength

McStas detectors/monitors

Monitoring: How and What do we monitor?



Nota bene:

weighted polarisation, *not* intensity

Polarisation along $\hat{h}$ *as fct. of wavelength*

Polarization monitors

- Available monitors:
  - `Pol_monitor.comp`: 0D
  - `PolLambda_monitor.comp`: 2D
  - `PolTOF_monitor.comp`: 2D
  - `MeanPolLambda_monitor.comp`: 1D

McStas precession algorithm

- Magnetic fields in McStas
- The challenge:
  - Fast beam/ray transport: # $rays > 10^6$
  - Unknown magnetic field and field strength
  - >1 Magnet → nested fields.

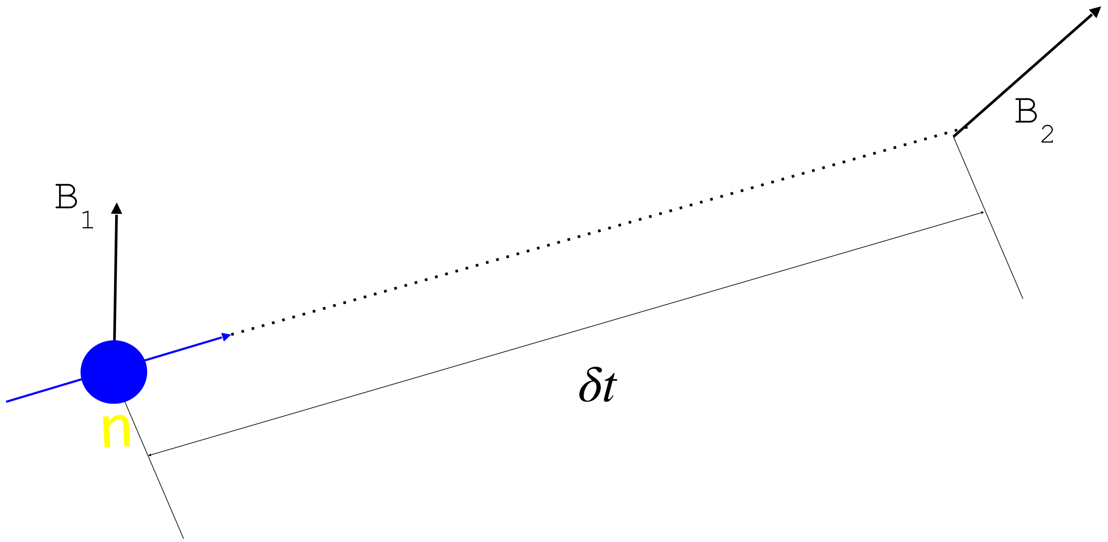
McStas precession algorithm

while $n_t < t_{target}$ **do**
- store neutron;
- sample magnetic field: $\mathbf{B}_1 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
- propagate neutron: $\delta t(< \Delta t)$;
- sample magnetic field: $\mathbf{B}_2 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
- **while** $|\mathbf{B}_1 - \mathbf{B}_2| > \delta B_{threshold}$ **do**
  - restore neutron;
  - $\delta t := \delta t/2$;
  - propagate neutron: $\delta t(< \Delta t)$;
  - sample magnetic field: $\mathbf{B}_1 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
- precess polarization: $\mathbf{P_n}$ by ω around $\frac{\mathbf{B}_1 + \mathbf{B}_2}{2}$;

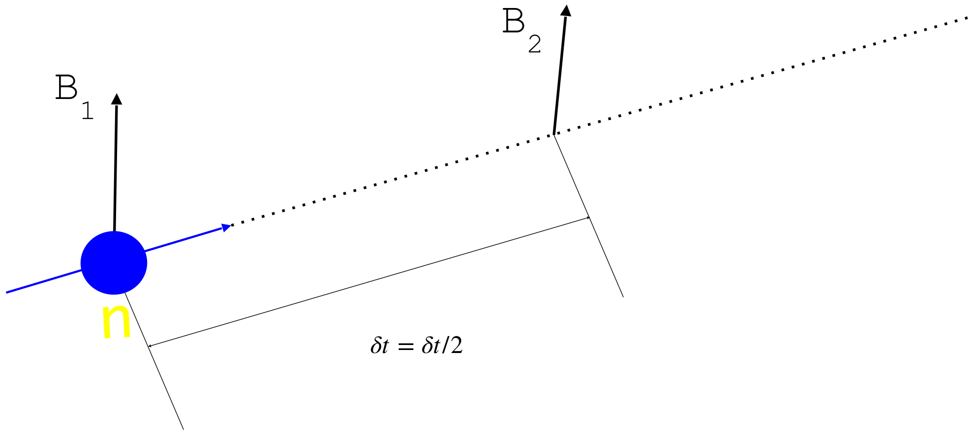
Algorithm 1: SimpleNumMagnetPrecession: Simplistic algorithm for tracking polarization of a Monte-Carlo neutron in a magnetic field. The neutron's state is stored as a position (n_x, n_y, n_z), a velocity $\mathbf{v}$, time n_t, and polarization vector $\mathbf{P_n}$.

From: Knudsen et.al., J. Neutron Research, 2014

McStas precession algorithm



McStas precession algorithm



McStas precession algorithm

while $n_t < t_{target}$ **do**

- store neutron;
- sample magnetic field: $\mathbf{B}_1 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
- propagate neutron: $\delta t (< \Delta t)$;
- sample magnetic field: $\mathbf{B}_2 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
- **while** $|\mathbf{B}_1 - \mathbf{B}_2| > \delta B_{threshold}$ **do**
  - restore neutron;
  - $\delta t := \delta t / 2$;
  - propagate neutron: $\delta t (< \Delta t)$;
  - sample magnetic field: $\mathbf{B}_1 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
- precess polarization: $\mathbf{P_n}$ by ω around $\frac{\mathbf{B}_1 + \mathbf{B}_2}{2}$;

Algorithm 1: SimpleNumMagnetPrecession: Simplistic algorithm for tracking polarization of a Monte-Carlo neutron in a magnetic field. The neutron's state is stored as a position (n_x, n_y, n_z), a velocity $\mathbf{v}$, time n_t, and polarization vector $\mathbf{P_n}$.

From: Knudsen et.al., J. Neutron Research, 2014

McStas precession algorithm

while $n_t < t_{target}$ **do**

  store neutron;

  sample magnetic f eld: $\mathbf{B}_1 = \mathbf{B}(n_x, n_y, n_z, n_t)$;

  propagate neutron: $\delta t(< \Delta t)$;

  sample magnetic f eld: $\mathbf{B}_2 = \mathbf{B}(n_x, n_y, n_z, n_t)$;

  while $|\mathbf{B}_1 - \mathbf{B}_2| > \delta B_{threshold}$ **do**

    restore neutron;

    $\delta t := \delta t/2$;

    propagate neutron: $\delta t(< \Delta t)$;

    sample magnetic f eld: $\mathbf{B}_1 = \mathbf{B}(n_x, n_y, n_z, n_t)$;

  precess polarization: $\mathbf{P_n}$ by ω around $\frac{\mathbf{B}_1 + \mathbf{B}_2}{2}$;

Algorithm 1: SimpleNumMagnetPrecession: Simplistic algorithm for tracking polarization of a Monte-Carlo neutron in a magnetic f eld. The neutron's state is stored as a position (n_x, n_y, n_z), a velocity $\mathbf{v}$, time n_t, and polarization vector $\mathbf{P_n}$.

From: Knudsen et.al., J. Neutron Research, 2014

McStas precession algorithm

while $n_t < t_{target}$ **do**
  store neutron;
  sample magnetic field: $\mathbf{B}_1 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
  propagate neutron: $\delta t (\boxed{< \Delta t})$;
  sample magnetic field: $\mathbf{B}_2 = \mathbf{B}(n_x, n_y, n_z, n_t)$;
  while $|\mathbf{B}_1 - \mathbf{B}_2| > \boxed{\delta B_{threshold}}$ **do**
    restore neutron;
    $\boxed{\delta t := \delta t / 2;}$
    propagate neutron: $\delta t (< \Delta t$
    sample magnetic field: $\mathbf{B}_1 =$
  precess polarization: $\mathbf{P_n}$ by ω a

Algorithm 1: SimpleNumMagnetPr
ing polarization of a Monte-Carlo ne
state is stored as a position $(n_x, n_y, n$
vector $\mathbf{P_n}$.

From: Knudsen et.al., J. Neutron Resea

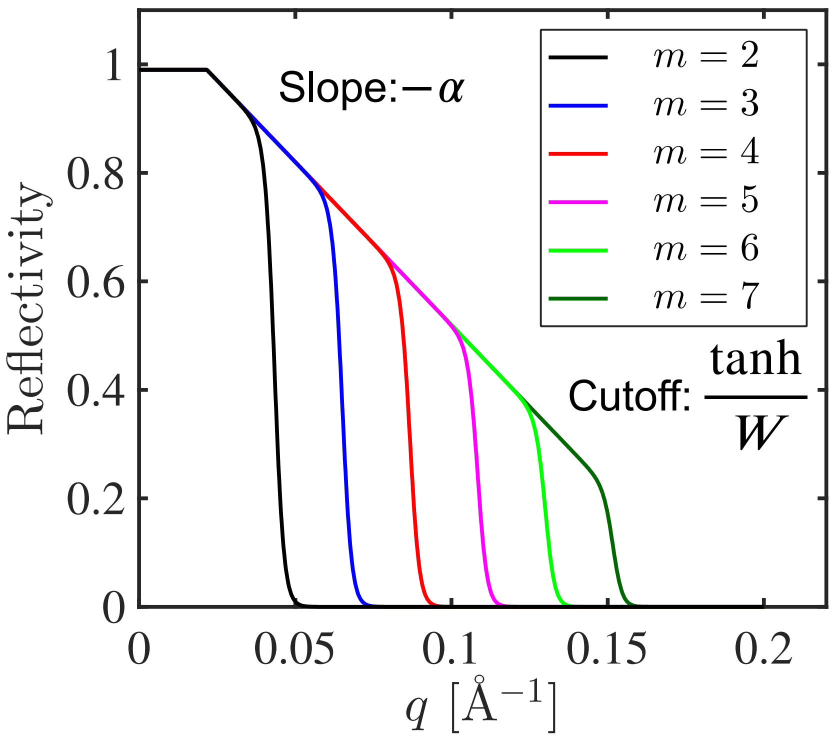
```
/*Threshold below which two magnetic fields are considered to be
 * in the same direction.*/
#ifndef mc_pol_angular_accuracy
#define mc_pol_angular_accuracy (1.0*DEG2RAD)
#endif

/*The maximal timestep taken by neutrons in a const field*/
#ifndef mc_pol_initial_timestep
#define mc_pol_initial_timestep 1e-5;
#endif
```

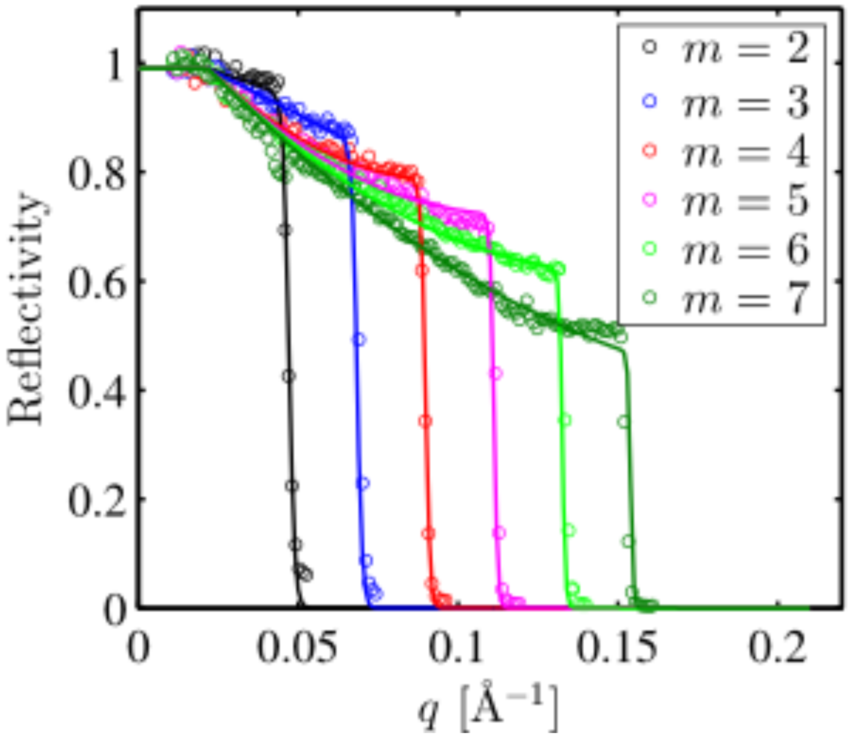
Reminder: Reflectivity curves in McStas

$$R(q) = \begin{cases} R_0 & \text{if } q < q_c \\ R_0(1 - \tanh((q - mq_c)/W))(1 - \alpha(q - q_c))/2 & \text{otherwise} \end{cases}$$

McStas standard model



McStas fitted model



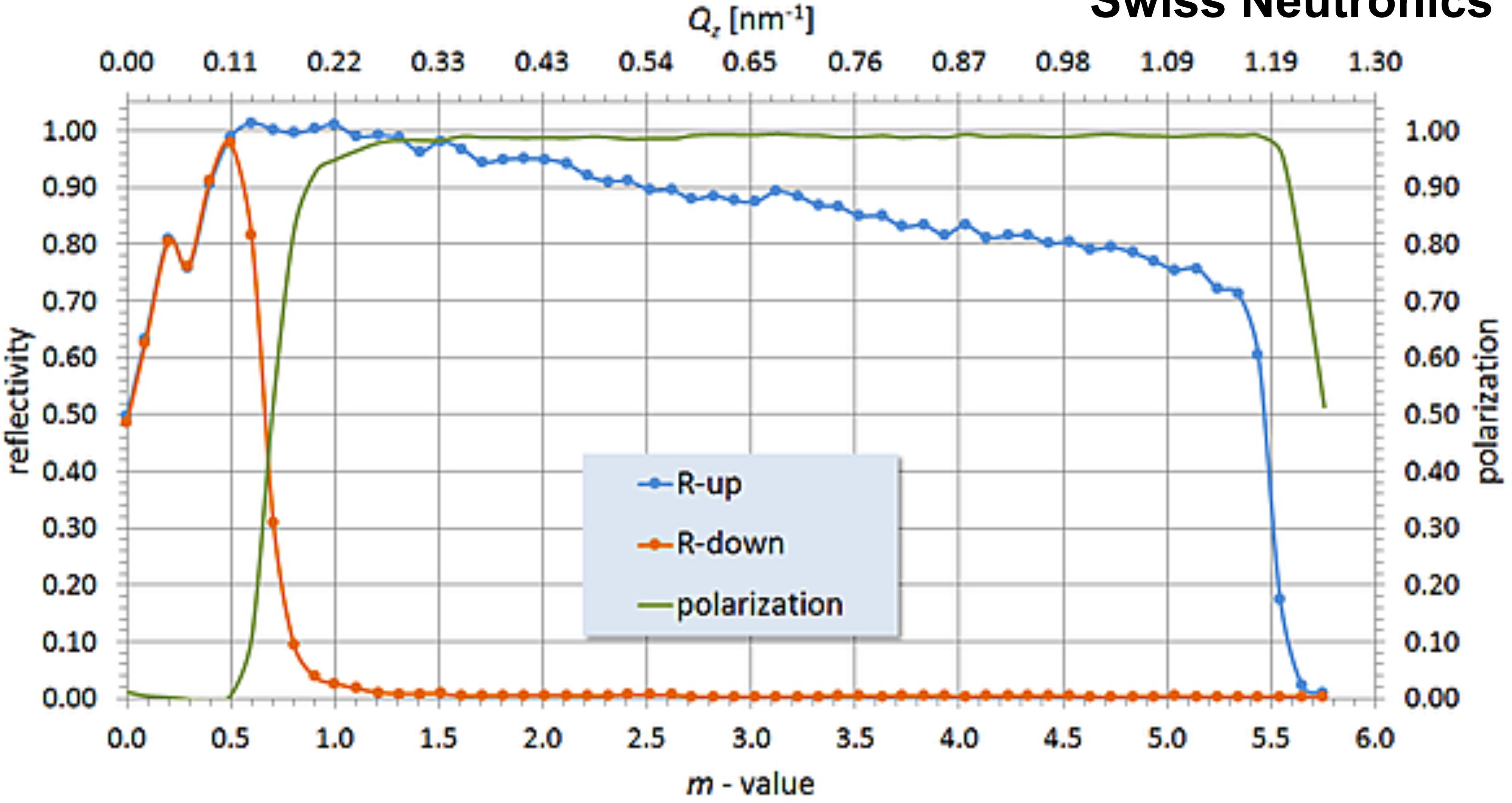
$\alpha = 0$
$W = 0$

Only m matters

Better mirrors available today

State-of-the-art Polarising Supermirrors (2019)

Swiss Neutronics



2025 ISIS McStas School

Depending on the component, several underlying “reflectivity” functions may be used:

```
// The McStas standard analytic parametrization of the reflectivity
void StdReflecFunc(double q, double *par, double *r);

// Reflectivity from a table using the routines in read_table-lib.
void TableReflecFunc(double q, t_Table *par, double *r);

// The McStas standard analytic parametrization of the reflectivity for
// double-side coated supermirror.
void StdDoubleReflecFunc(double q, double *par, double *r);

// “Extended” reflectivity fct. with 2nd order ‘beta’ parameter in  mc_pol_par
void ExtendedReflecFunc(double mc_pol_q, double *mc_pol_par, double *mc_pol_r);
```

McStas polarization components

Magnetic fields:

- Pol_FieldBox.comp
- Pol_Bfield.comp
- Pol_Bfield_stop.comp
- Pol_triafield.comp
- (Pol_tabled_field 3.x)

Monitors:

- Pol_monitor.comp
- MeanPolLambda_monitor.comp
- PolLambda_monitor.comp
- PolTOF_monitor.comp

Contrib:

- Foil_flipper_magnet.comp

Optics:

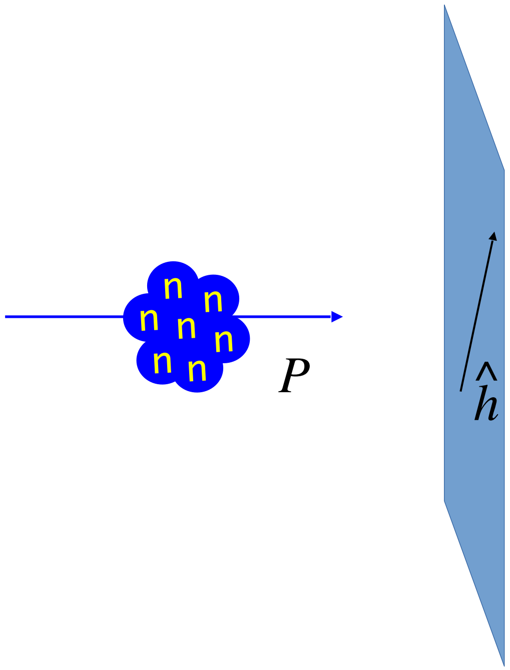
- Monochromator_pol.comp
- Pol_bender.comp
- Pol_guide_mirror.comp
- Pol_guide_vmirror.comp
- Pol_mirror.comp
- Transmission_polarisatorABSnT.comp
- Pol_bender_tapering.comp

Idealized components:

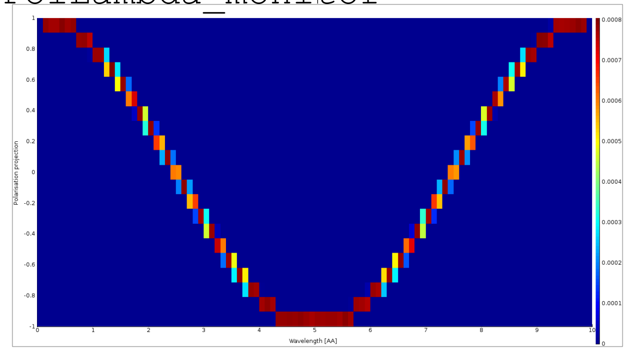
- PolAnalyser_ideal.comp
- Pol_SF_ideal.comp
- Pol_pi_2_rotator.comp
- Set_pol.comp

McStas polarization monitors

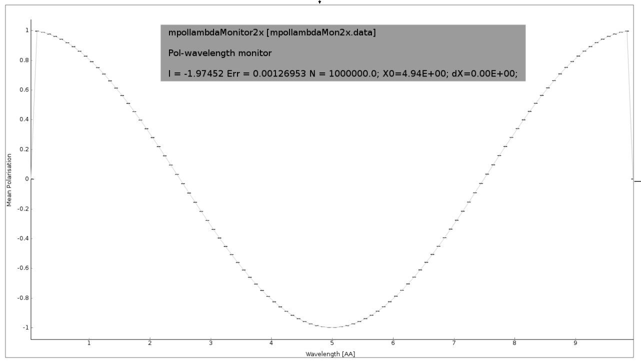
Monitors



`PolLambda_monitor`



`MeanPolLambda_monitor`

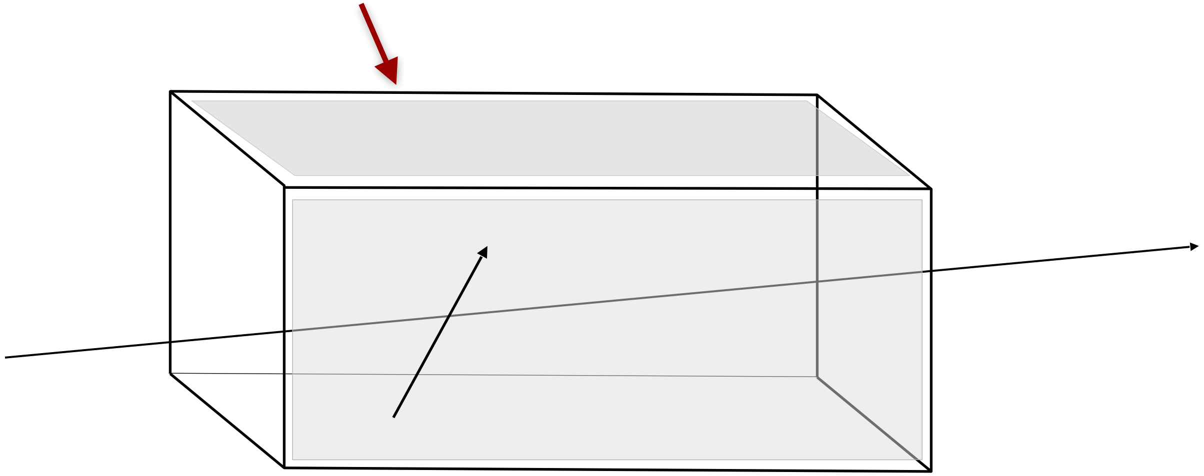


`Pol_monitor`

$$P \parallel \left(m_x, m_y, m_z\right) = 0.87$$

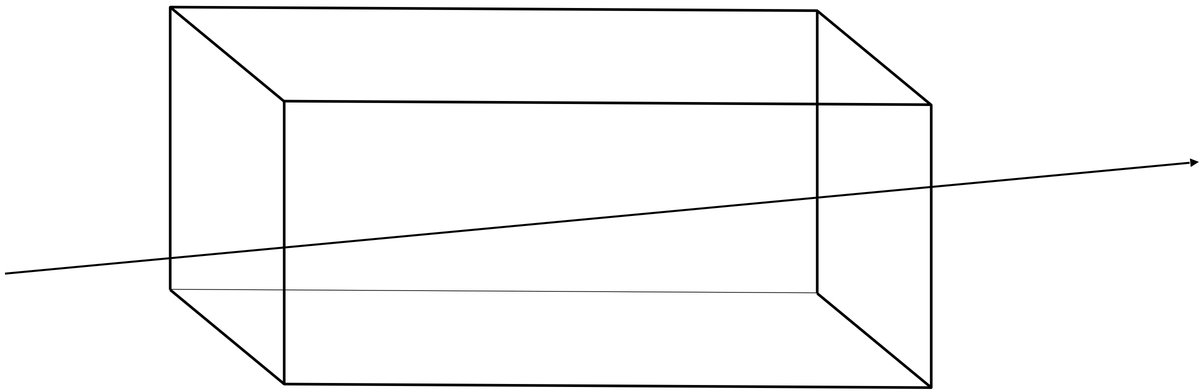
McStas magnetic fields

- `Pol_constBfield.comp`
- Single constant Magnetic field in a “box”.
- - user may specify a wavelength to flip.
  - “blocking walls”



McStas magnetic fields

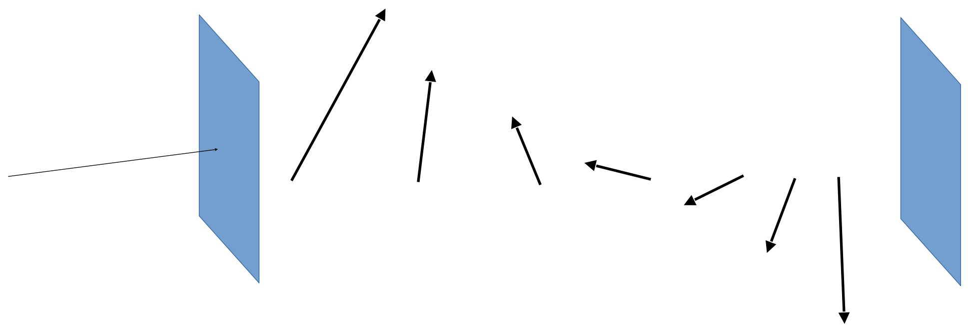
- `Pol_FieldBox.comp`
- Single Magnetic field in a “box”
- Constant or tabled magnetic fields



McStas magnetic fields

- `Pol_Bfield.comp`
- `Pol_Bfield_stop.comp`

```
- Entry/Exit contruction allows for nested magnetic field descriptions.
- Any magnetic field through user supplied c-function
- Tabled magnetic fields
```



Standard field types:

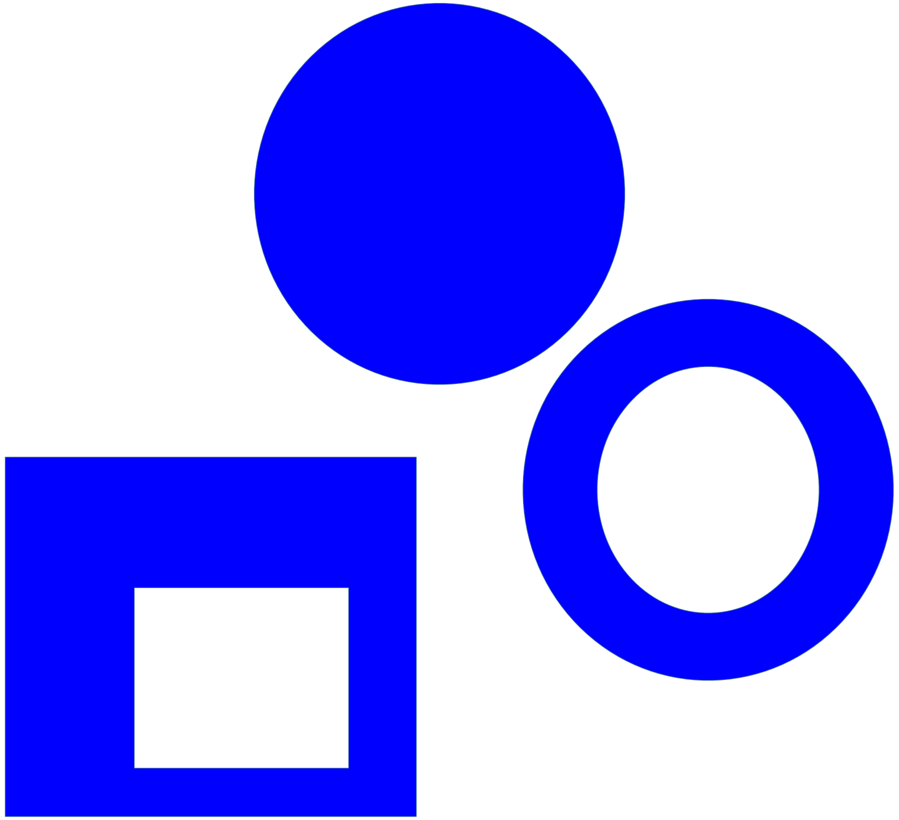
- Constant field
- Rotating field
- Gradient field
- “Majorana” type field

Plus user-defined fields (McStas 2.x only)

See pol-lib.c in share/ and the examples

Windows can be many shapes

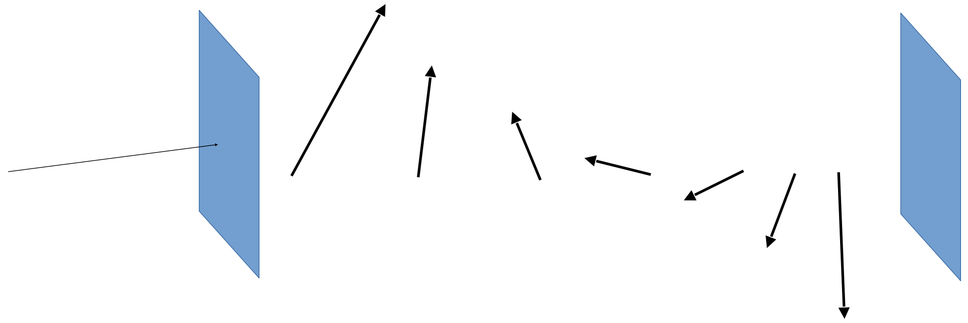
B-Fields: constant, functional, tabled, ... in more general shapes



McStas magnetic fields

- `Pol_Bfield.comp`
- `Pol_Bfield_stop.comp`

```
- Entry/Exit contruction allows for nested magnetic field descriptions.
- Any magnetic field through user supplied c-function
- Tabled magnetic fields
```



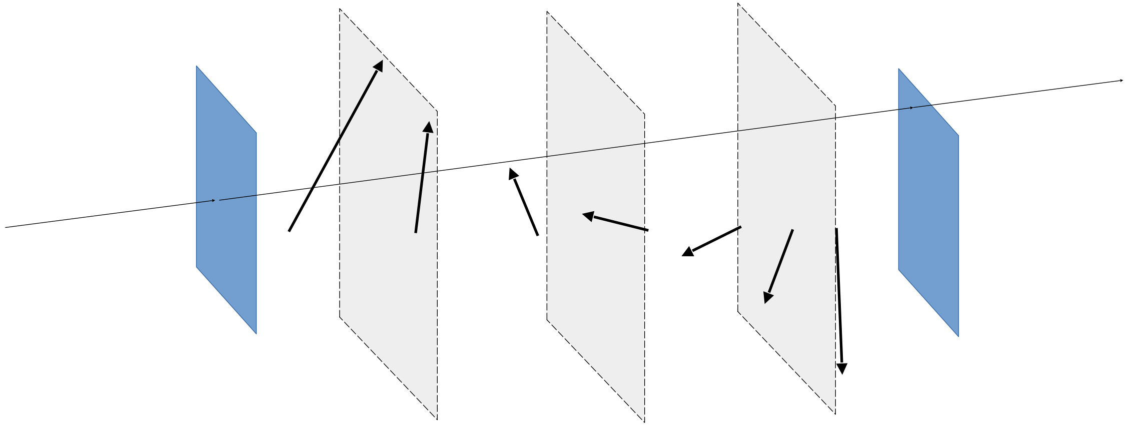
Standard field types:

- Constant field
- Rotating field
- Gradient field
- “Majorana” type field

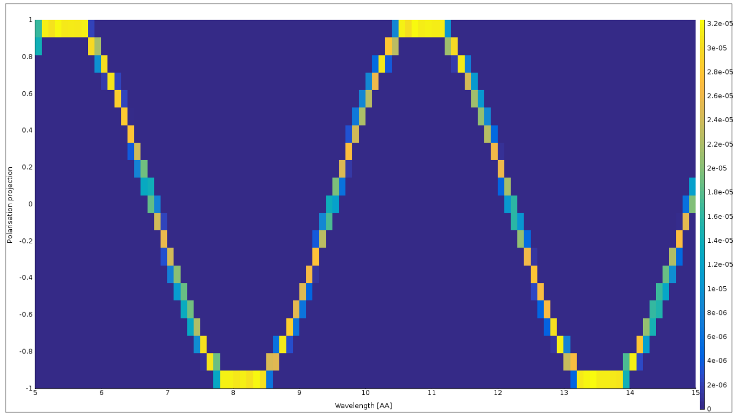
Plus user-defined fields (McStas 2.x only)

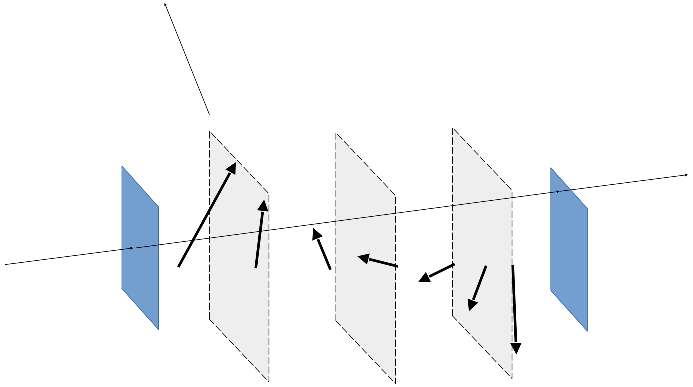
See pol-lib.c in share/ and the examples

Pol_monitors along the way…

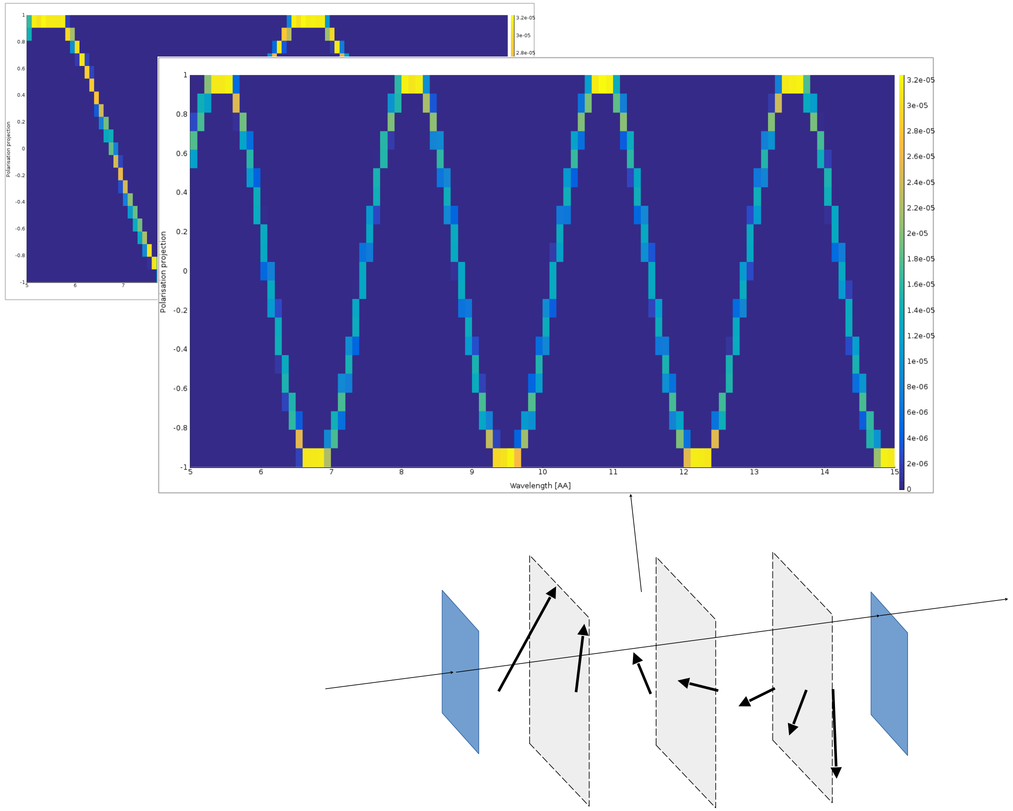


Pol_monitors along the way…

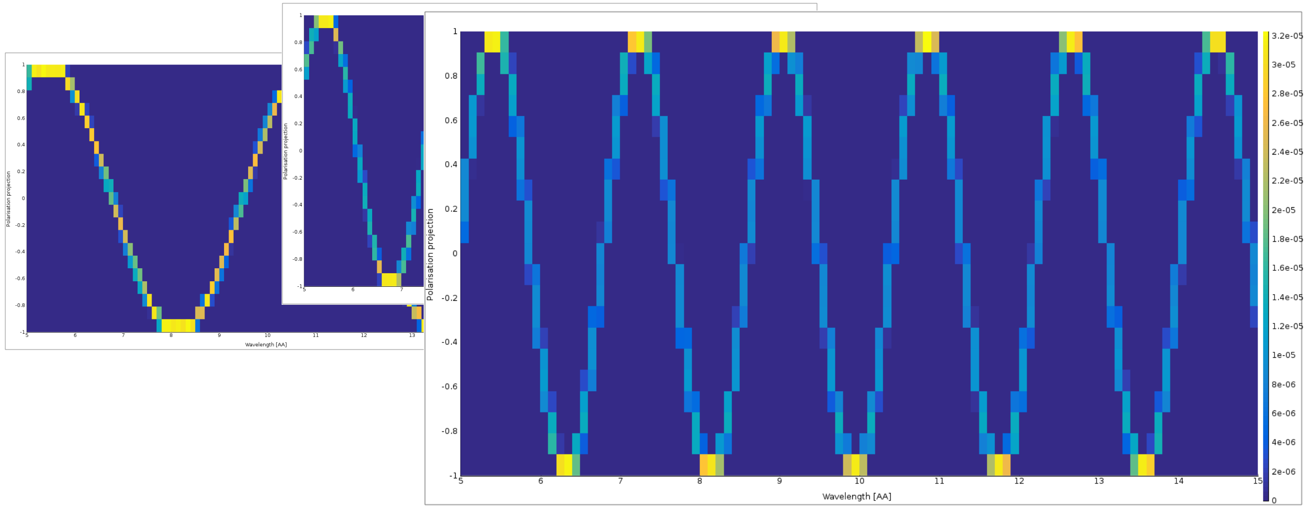


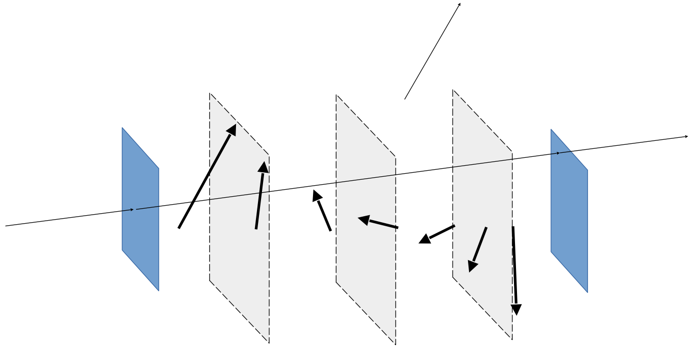


Pol_monitors along the way…

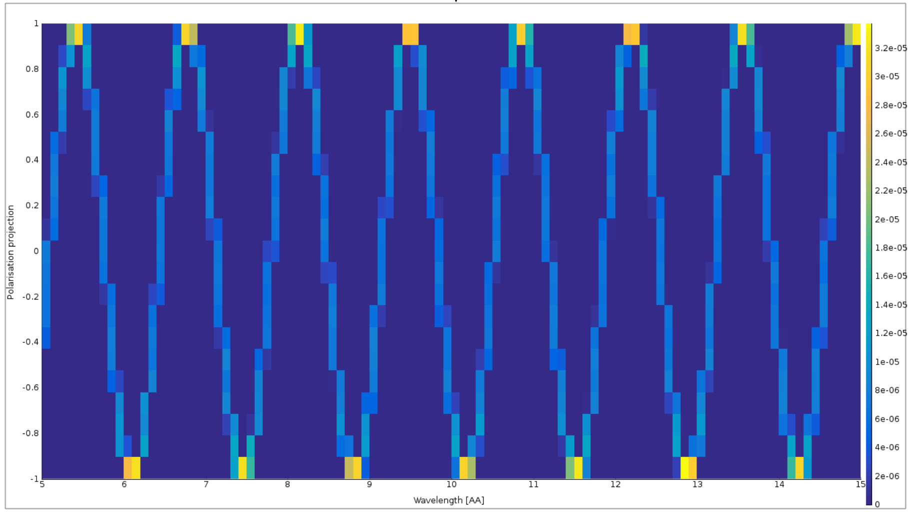


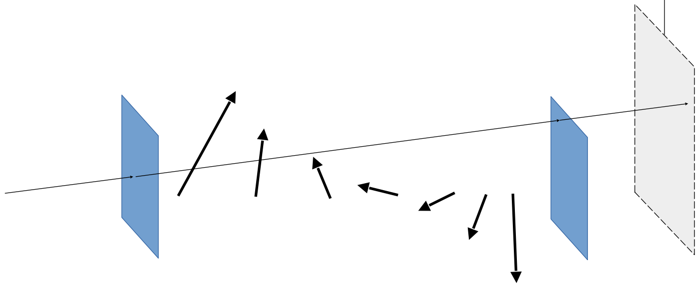
Pol_monitors along the way...



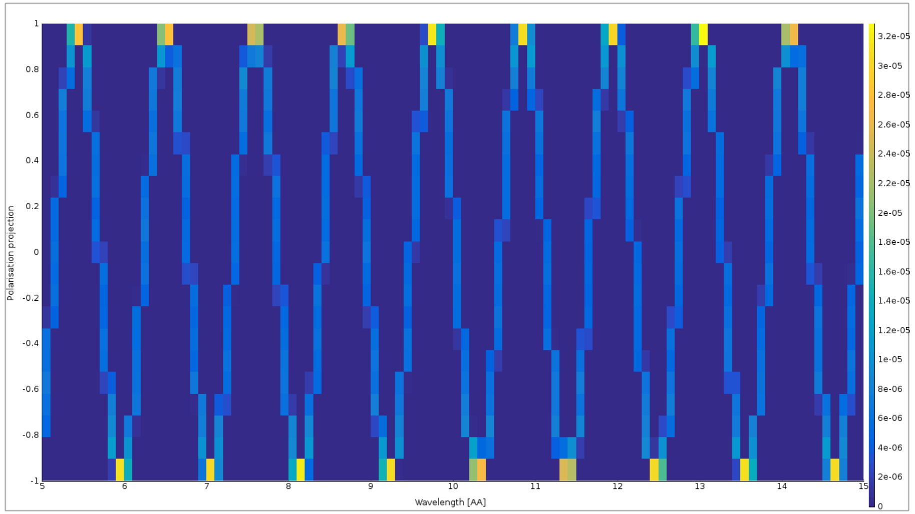


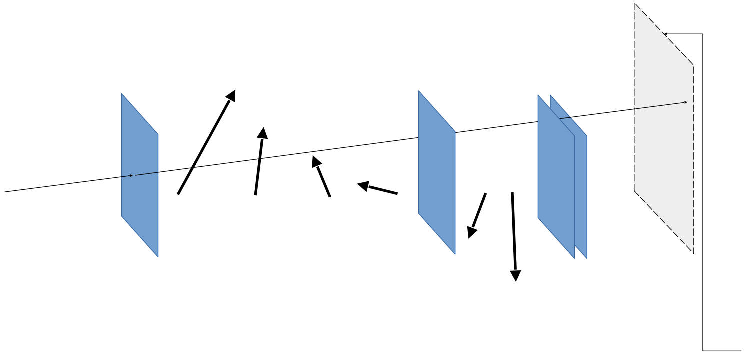
Nested fields





Nested fields





Simple McStas sample component

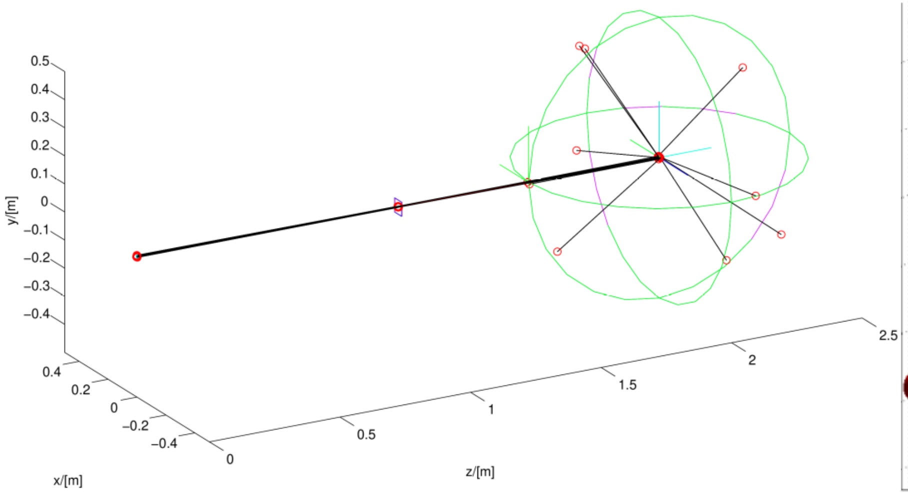
Incoherent.comp has SF / NSF solution

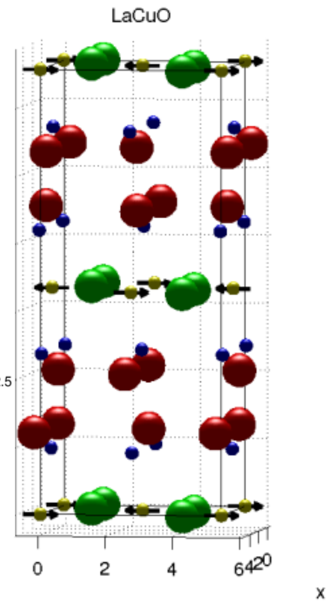
```
/* Polarisation part (1/3 NSF, 2/3 SF) */
sx *= -1.0/3.0;
sy *= -1.0/3.0;
sz *= -1.0/3.0;

SCATTER;
```

McStas sample component

Magnetic single crystal

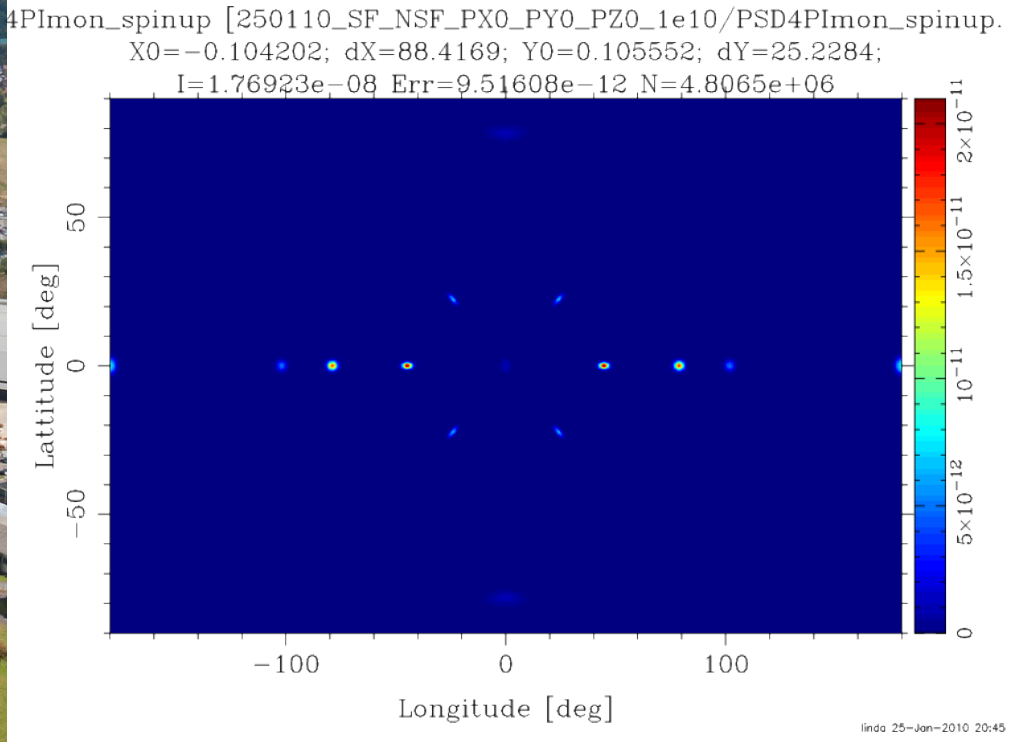


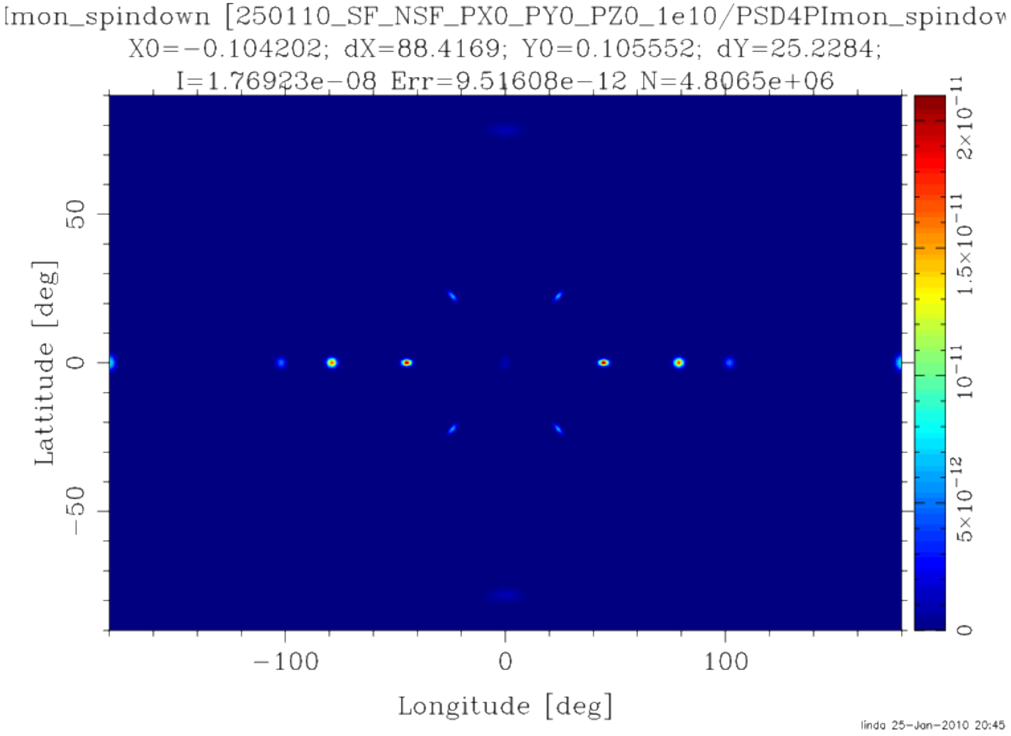


index	iontype	x	y	z	b_{coh}[fm]	g_S	S_x	S_y	S_z	g_L	L_x	L_y	L_z
1	Cu2+	0.5	0.5	0	7.718	2	0	-0.5	0	0	0	0	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

McStas sample component

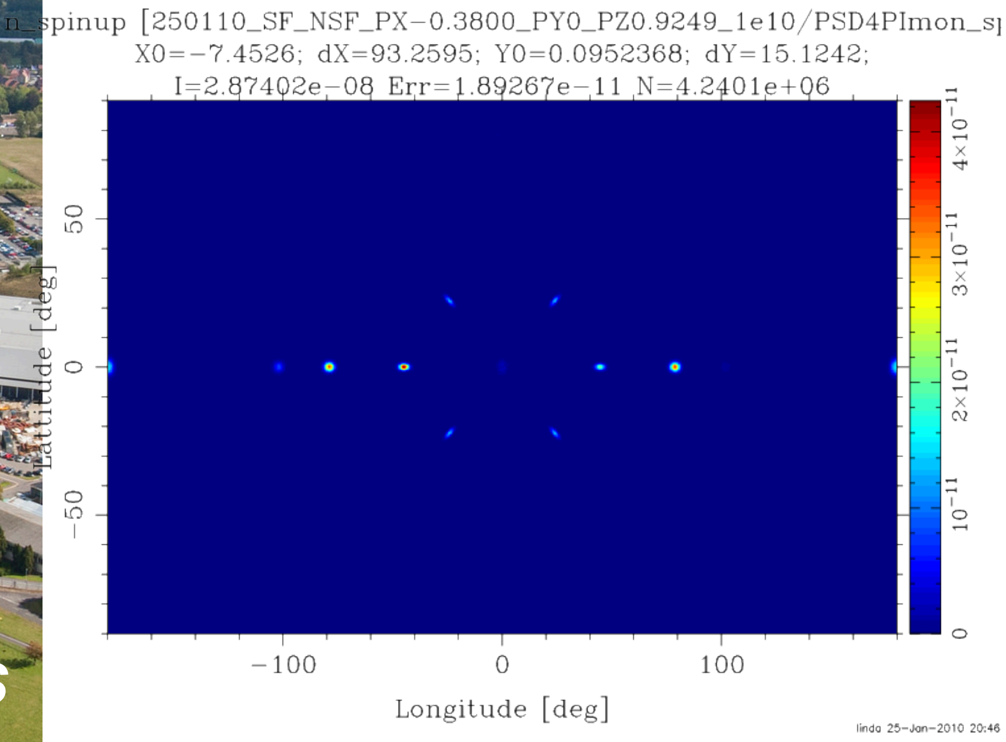
Magnetic single crystal – **Unpolarized** beam

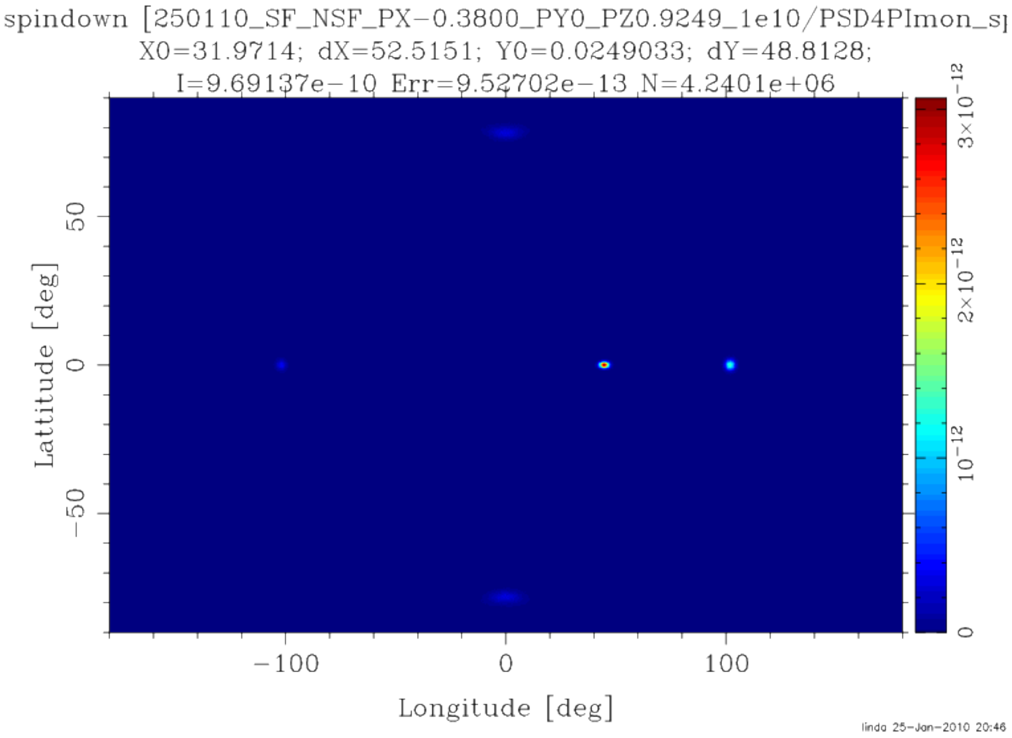




McStas sample component

Magnetic single crystal – **Polarized** beam





2025 ISIS McStas School

McStas sample component

Magnetic single crystal

The magnetic scattering cross-section for a sample with localised spin+orbital angular moment $g\mathbf{J} = (g_S + g_L)\mathbf{J} = 2\mathbf{S} + \mathbf{L}$ is:

$$\frac{\mathrm{d}^2\sigma}{\mathrm{d}\Omega_\mathrm{f}\mathrm{dE}_\mathrm{f}} = \frac{k_f}{k_i} \sum_{i,f} P(\lambda_i) \left| \langle \lambda_f \mid \sum_j e^{i\mathbf{Q}\cdot\mathbf{d}_j} U_j^{\sigma_i\sigma_f} \mid \lambda_i \rangle \right|^2 \delta(\hbar\omega + E_i - E_f)$$

where $\mid \lambda_i\rangle$ and $\langle\lambda_f \mid$ are the initial and final states of the sample with energies E_i and E_f respectively, $P(\lambda_i)$ is the distribution of initial states and

$$U_j^{\sigma_i\sigma_f} = \langle \sigma_f \mid b_j - m_j \mathbf{J}_{\perp j} \cdot \boldsymbol{\sigma} \mid \sigma_i \rangle$$

where $\mid \sigma_i\rangle$ and $\langle \sigma_f \mid$ are the initial and final spin states of the neutron, and $\boldsymbol{\sigma}$ are the Pauli spin matrices working on the neutron state.

From: G. Shirane et.al. ,"Neutron Scattering with Triple-Axis Spectrometer", Cambridge Univ. Press, 2002

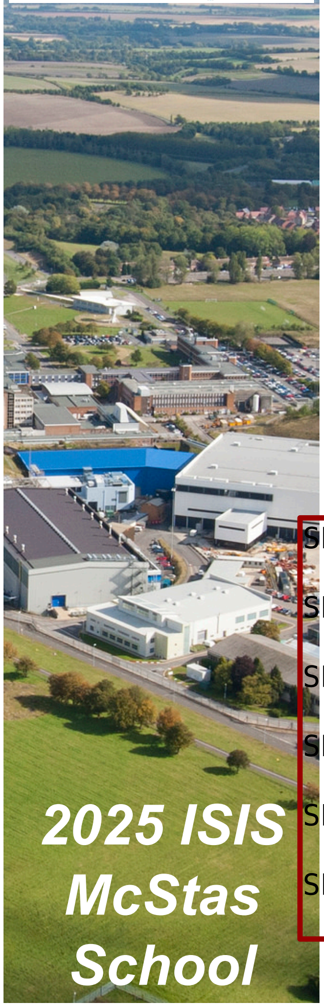
McStas sample component

Magnetic single crystal

If $\mathbf{P} = P(\xi, \eta, \zeta) = P\hat{\zeta}$.Thus, the matrix elements of $U^{\sigma_i \sigma_f}$ can now be written

$$
\begin{aligned}
U^{++} &= b - mJ_{\perp\zeta} \\
U^{--} &= b + mJ_{\perp\zeta} \\
U^{+-} &= -m\left(J_{\perp\xi} + iJ_{\perp\eta}\right) \\
U^{+-} &= -m\left(J_{\perp\xi} - iJ_{\perp\eta}\right)
\end{aligned}
$$

where $m = \frac{r_0 \gamma}{2} g f(\mathbf{Q})$ with r_0 the classical electron radius, $\gamma = 1.913$, g the Landé splitting factor and $f(\mathbf{Q})$ the magnetic form factor of a particular ion in the sample.



Example instruments:

Magnetic.instr:

Test_Magnetic_Constant.instr

Test_Magnetic_Majorana.instr

Test_Magnetic_Rotation.instr

Test_Magnetic_Userdefined.instr (2.x only)

Test_single_magnetic_crystal.instr (2.x only)

SE*.instr:

SEMSANS_Delft.instr

SEMSANS_instrument.instr

SESANS_Delft.instr

SE_example.instr

SE_example2.instr

Pol.instr:

Test_Pol_Bender.instr

Test_Pol_Bender_Vs_Guide_Curved.instr

Test_Pol_FieldBox.instr

Test_Pol_Guide_Vmirror.instr

Test_Pol_Guide_mirror.instr

Test_Pol_MSF.instr

Test_Pol_Mirror.instr

Test_Pol_SF_ideal.instr

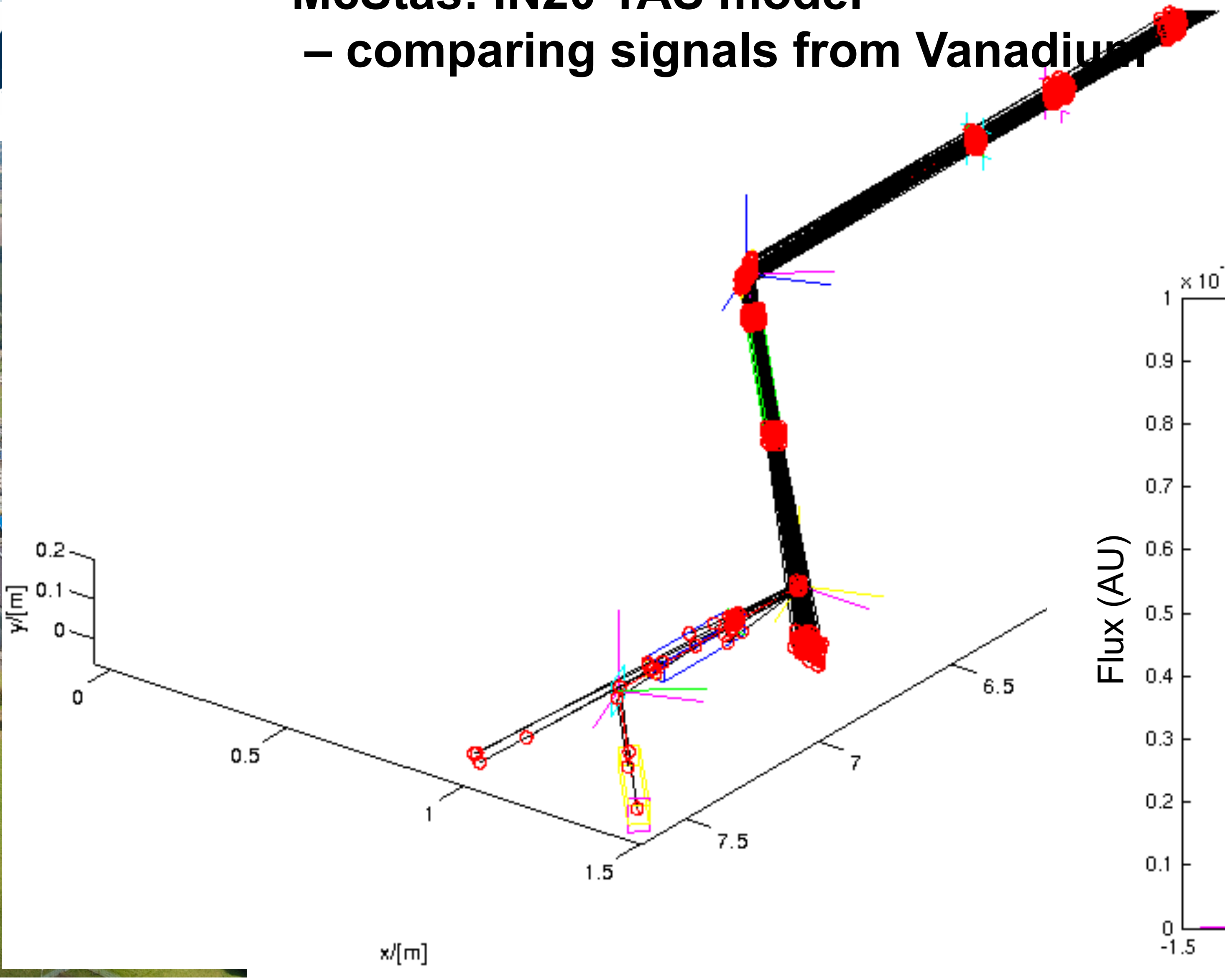
Test_Pol_Set.instr

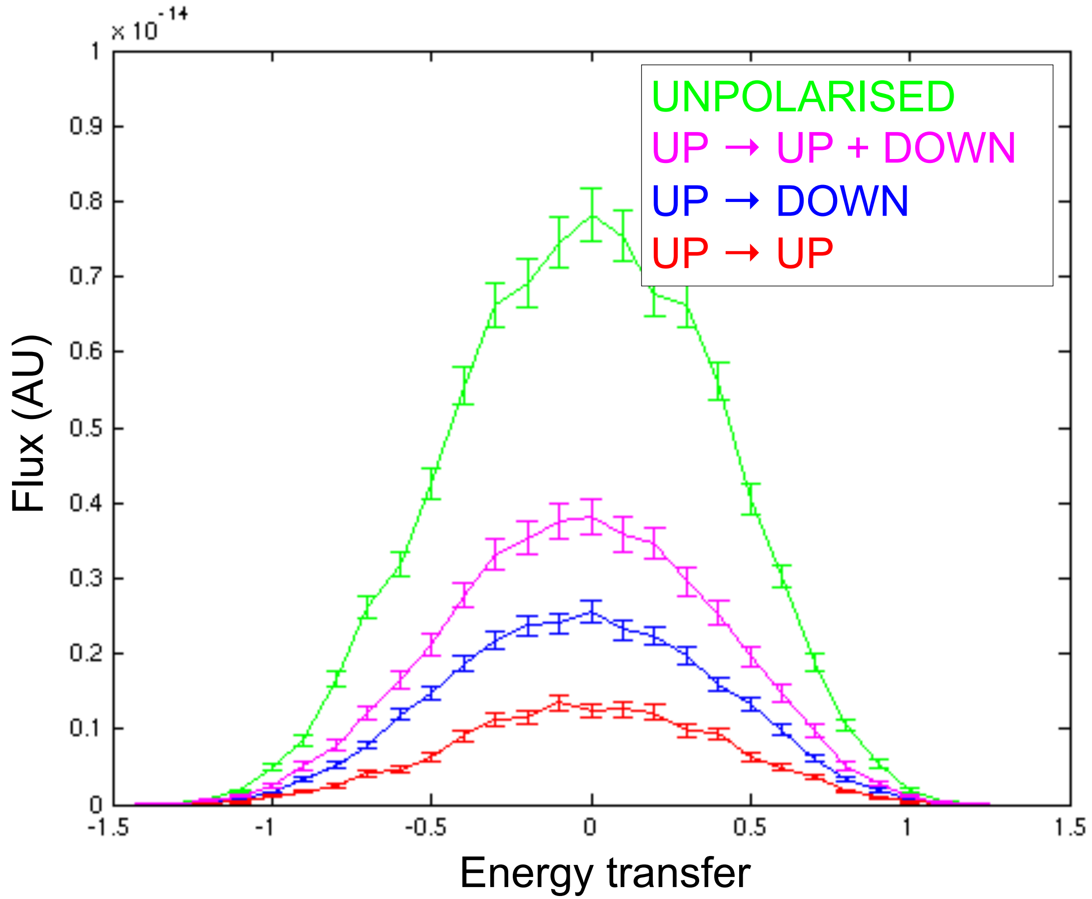
Test_Pol_Tabled.instr

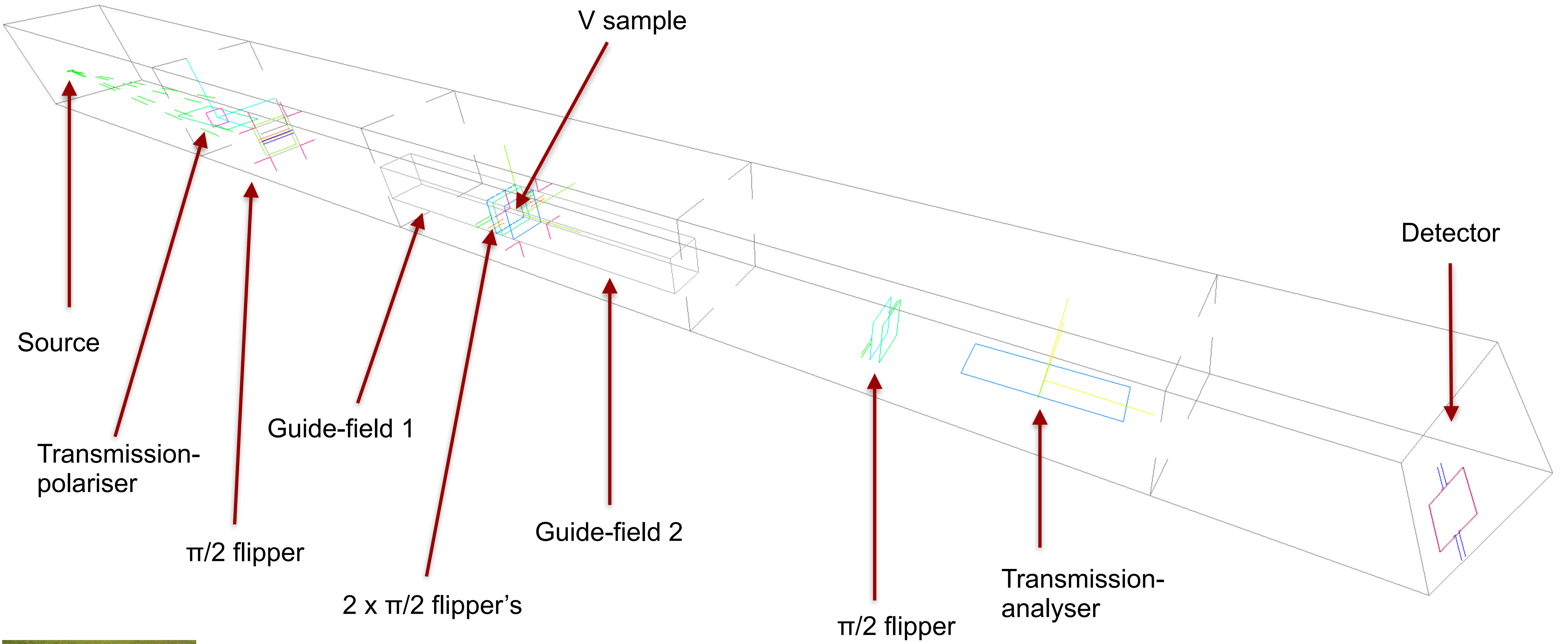
Test_Pol_TripleAxis.instr

A copy of the files from McStas 3.5.x is placed in the ‘instrs’ folder

McStas: IN20 TAS model
– comparing signals from Vanadium

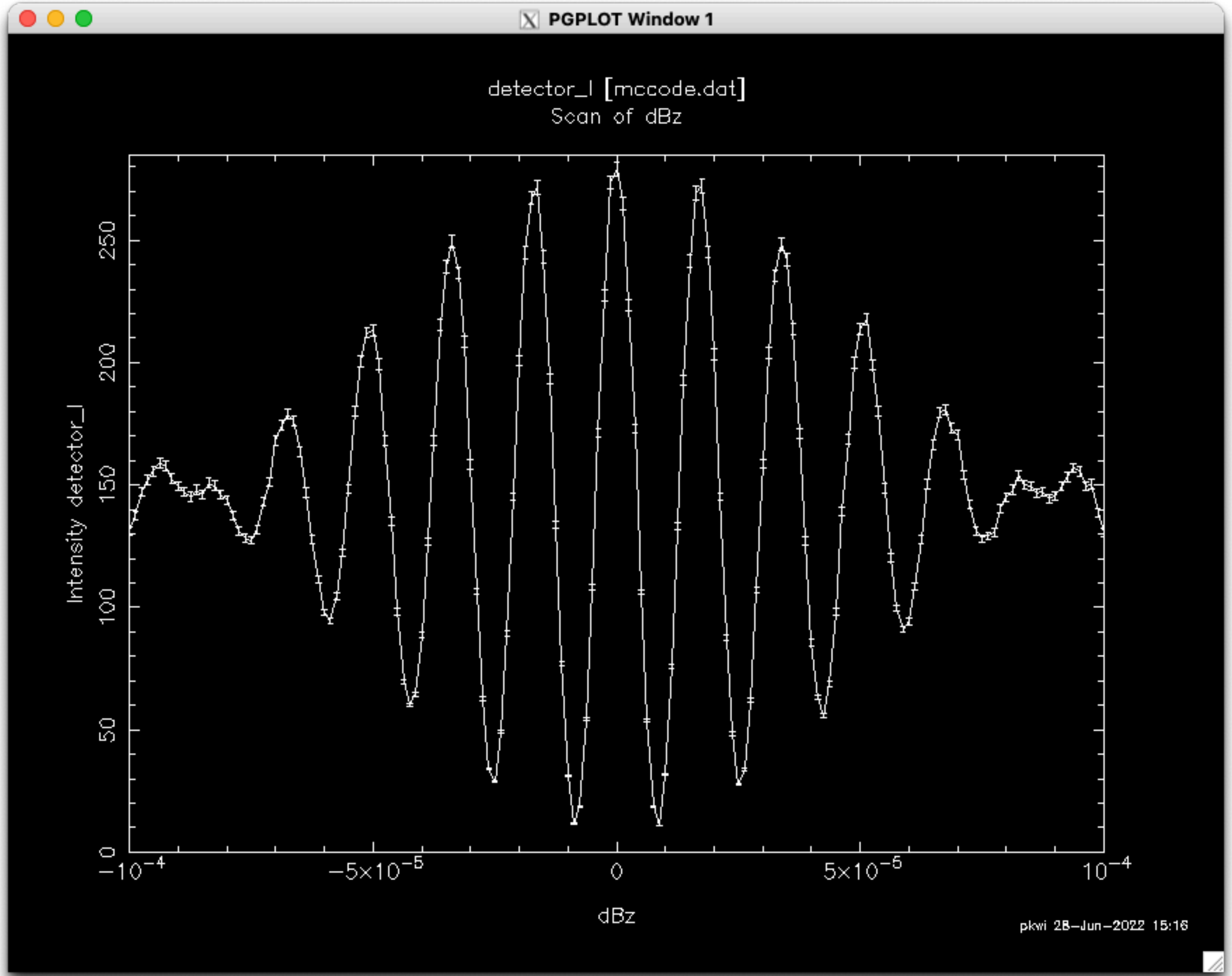





V sample
Detector
Source
Guide-field 1
Transmission-
polariser
π/2 flipper
Guide-field 2
2 x π/2 flipper's
π/2 flipper
Transmission-
analyser

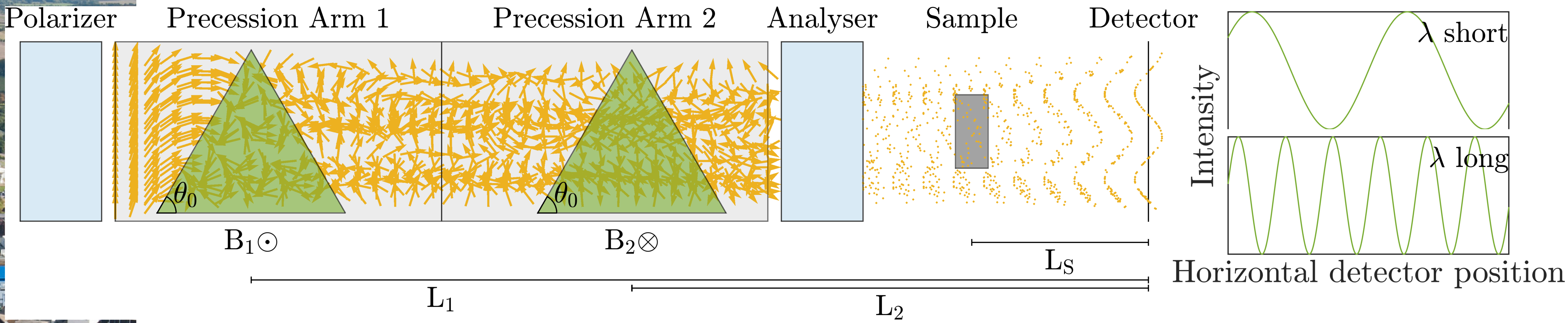
School

Tuning of guide-field 1 around its value allows us to see spin-echo:



Highlight:
McStas example SEMSANS

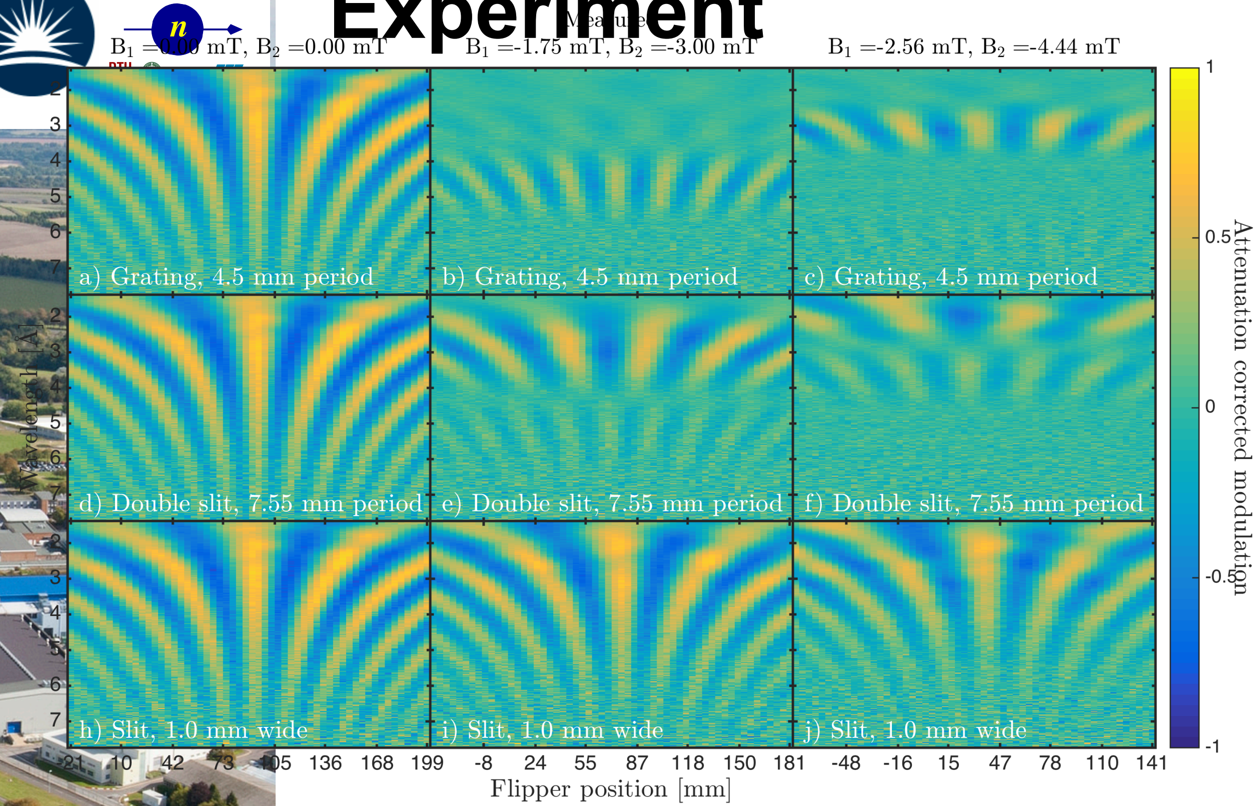
Courtesy: M. Sales et.al.



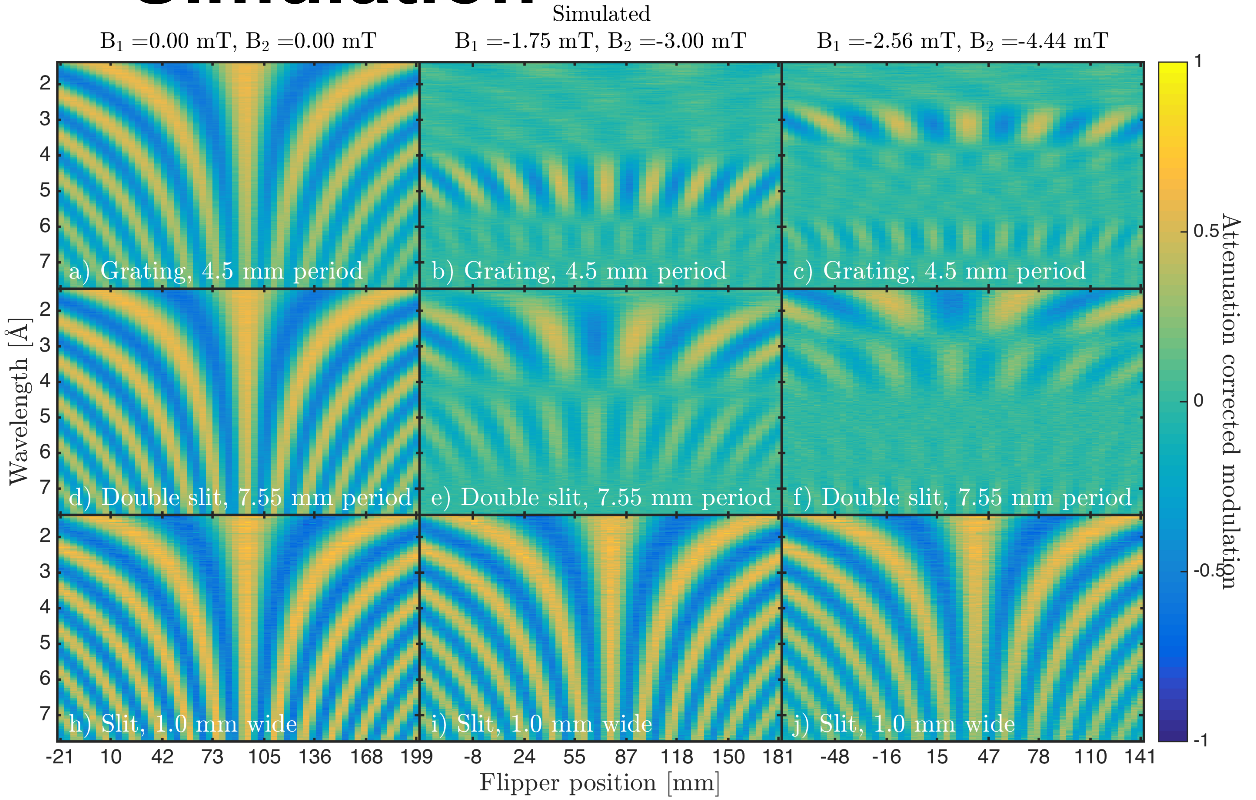
2025 ISIS McStas School

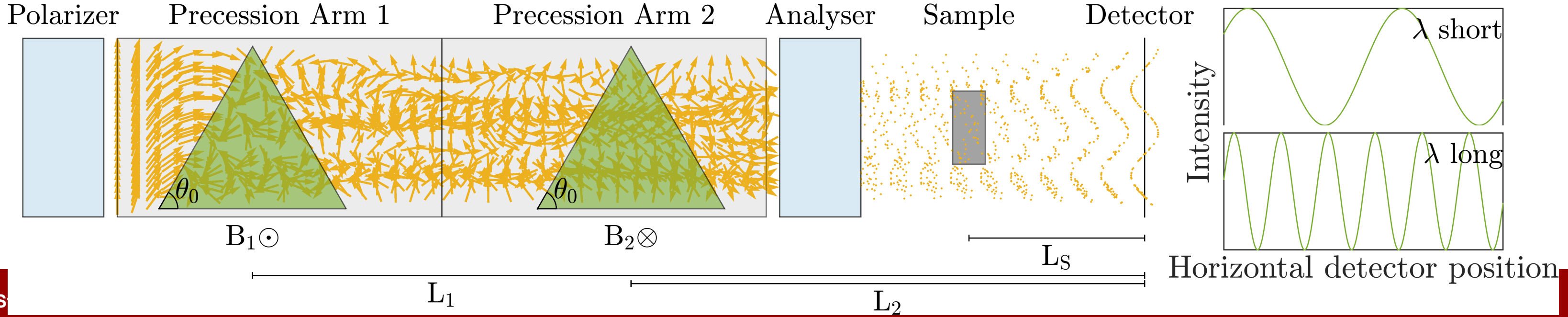
Highlight:
McStas example SEMSANS

Experiment



Simulation





Gotcha’s

- Polarisation docs in manual are **out of date** wrt. the current code/algorithm…
- For now, **don’t combine gravity and polarisation!**
  (Guide_gravity and Elliptic_guide_gravity can currently not be used within field regions.)
- Pol_Bfield placement can be confusing depending on choice of geometry-parts…
- McStas 3.x does not support
  - User-defined field functions
    (No function pointers on GPU… code-generator may rescue us later)
  - Having optics within **tabled** fields
    (tabled fields are for now only available in Pol_tabled_field which can not work in ” concentric geometries”)
- GitHub issues exist and these things will eventually all be fixed!